

* most **** important topic *

multithreading

multitasking

multiprocessing

multithreadings

8084329452

difference b/w
Thread and process

How to create thread
in java

Verification

By thread class

By Runnable class (methods)

Life cycle of Thread

How to create thread using
thread class

extending thread class

implementing Runnable interface

Thread Execution cases

S.T.S.Th

S.T.M.Th

M.T.S.Th → Not possible

M.T.M.Th

Thread class

methods

Constructors

Thread class methods part - ①

Thread Naming methods

getName(), setName()

Current thread

isAlive

~~##~~ Daemon Threads

~~##~~ use of daemon thread

~~##~~ Nature of daemon thread

~~##~~ Life of daemon thread

~~##~~ Program Case of daemon thread

~~##~~ Thread priorities in java

~~##~~ MIN_priorities

~~##~~ NORM_priorities

~~##~~ MAX_priorities

~~##~~ set priorities

~~##~~ get priorities

~~##~~ nature

~~##~~ sleep() methods

~~##~~ sleep(long ms)

~~##~~ sleep(long ms, int nano)

~~##~~ uses

~~##~~ calling run() methods instead of start() methods

yield() methods in java

Working

java 5 . # java 6

prototype

example

join methods() in multithreading

Difference b/w sleep() yield() and join method

Interrupt() , Interrupted() and isInterrupted() methods

Introduction to synchronization
classification of synchronization

Daemon thread in java

ये Background में run करते हैं जो main thread की service provide करते हैं।

which run in the background of another thread

it provides service to the thread

ex:- Garbage collector, finalizer

Attached Listener, signal
dispatch etc.

*→ spelling checker: ये Background में run करता है और check करता है

कि जो नि हम type कर रहे हैं वो सही है या नहीं

* Methods :-

i) setDaemon

public final void setDaemon(boolean b)

ii) isDaemon

public final boolean isDaemon()


```
class Test extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        sop("child thread")
```

```
    }
```

Daemon thread

```
public static void main(String[] arg)
```

```
{
```

```
    sop("main thread");
```

```
    Test t = new Test();
```

```
    t.setDaemon(true);
```

```
    t.start();
```

```
}
```

main thread

Important case

#⇒ Daemon thread जब नि हम
create करते हैं उसे हमें thread
start करने पहले ही create

करना पड़ेगा

We have to create Daemon
thread before starting
the thread

if we create Daemon thread
after starting it will

throw

run time exception

Illegal thread exception

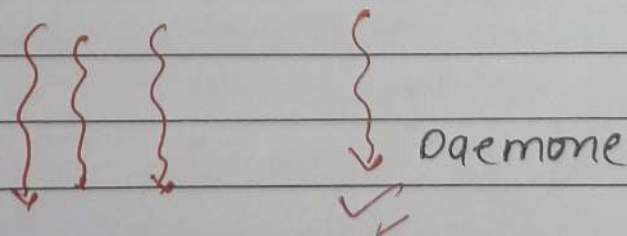
*→ We can't create main methods
as a Daemon methods thread

*** Daemon thread ki life depend
करता है जिस Thread के Background
में execute होता है।

** Daemon Thread ki properties main Thread
(जिसके Background में काम कर रहा है)

उससे Inherit (extends) होगी।

*** JVM जोर



⇒ JVM not depend on daemon- Last
→ threads

**

Demon thread की

priority हमें Low रखनी चाहिए,

But we can change the

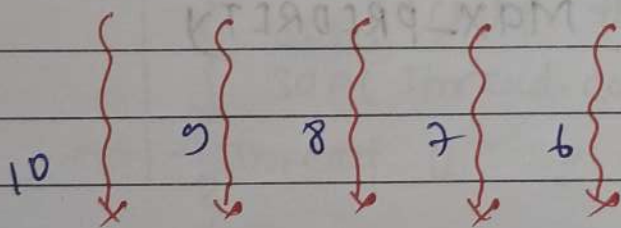
priority of Demon threads

* Thread priorities in java *

जब मैं नए Thread create करते तो

JVM उसकी priorities set करवा देता है।
* * * *

इसी priorities के basis पर JVM
processor allocate करता है।



xxx * JVM provides the priorities to

each thread and according to
these priorities JVM allocate

the processor

* * * priorities are represented in the number

form
range :- (1 to 10)

1 → MIN-PRIORITY

5 → NORM-PRIORITY

10 → MAX-PRIORITY

x x x Below are not priorities

0, 11, 10

minimum-priority

midium-priority

Low priority

Normal-priority

Maximum-priority

High-priority

2
exist
10
10

methods priority

⇒ public final void setPriority (int value) x

⇒ public final int getPriority () x

→ class test extends Thread {

public void run() {

{ sop("child thread")

sop(Thread.currentThread().getPriority());

Thread: 0

}

5

p.s.v.m (str[] args) {

default = 5

test t = new test

t.start();

}

}

**** Default Nature :-**

priority are inherited

from parent thread

By default main thread priorities

(5)

xxx if priority value is not in

between (1) to (10)

then it will throw exception

Illegal Argument Exception

*** priorities depends on platform

*** Windows does not support priorities

Sleep methods

class Thread

{

* public static native void sleep(long ms)

throws

InterruptedException

Native

means

इस sleep() का

implementation किसी दूसरे language

के अंदर कि गई है।

* public static void sleep(long milli, int nano)

throw

InterruptedException

ये दोनों methods static हैं इसलिए इन दोनों

methods को thread class के through

Direct call कर सकते हैं।

example :-

class Test extends Thread

{

public static void main(String arg)

{

for (int i = 1; i <= 5; i++) {

try {

Thread.sleep(1000)

System.out.println(i);

} catch (Exception e) {

System.out.println(e);

}

}

main

Thread

- 1
- 2
- 3
- 4
- 5

example :-

```
class Test extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            try {
```

```
                Thread.sleep(1000);  
                System.out.println(i);
```

```
            } catch (Exception e) {
```

```
                System.out.println(e)
```

```
            }
```

```
        }
```

```
    public static void main (String arg) {
```

```
        Test t1 = new Test();
```

```
        t1.start();
```

```
        Test t2 = new Test();
```

```
        t2.start();
```

```
    }
```

main
thread

Thread_0

** if value of millisecond is negative $\rightarrow$ illegalArgument

Exception thrown

** if value of nanosecond is not in range of 0-999999

$\rightarrow$ illegalArgument Exception thrown

** sleep() method pause the execution of thread

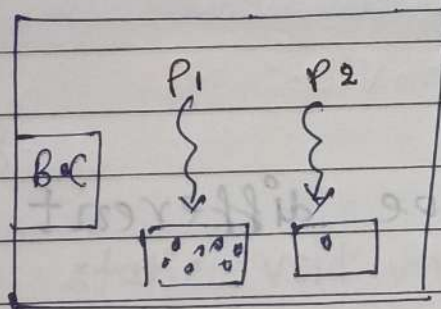
calling run() methods instead
of start() methods #

* yield() methods in java

multithreading *

⇒ yield() :- stop the current executing thread and gives a change to other thread for execution :-

prog :-



⇒ ये रुक stop हो जाता है। दूसरे person की, chance provide करता है कि पहले आय execute हो जाओ,

Working :-

java 5 :- Internally use sleep() methods

java 6 :- Thread provide the hint to the thread scheduler then it depend on Thread scheduler to Accept or ignor

prototype :-

```
public static native void yield();
```

Native : $\Rightarrow$ Implementation in
another language

** output may be different

class Test extends Thread

→ {

public void run() {

for(int i=1; i<5; i++) {

System.out.println(Thread.currentThread().getName() + "-t" + i);

}

public static void main(String[] args) {

Test t = new Test();

t.start();

for(int i=1; i<5; i++) {

Thread.yield();

System.out.println("main thread" + "-t" + i);

}

}

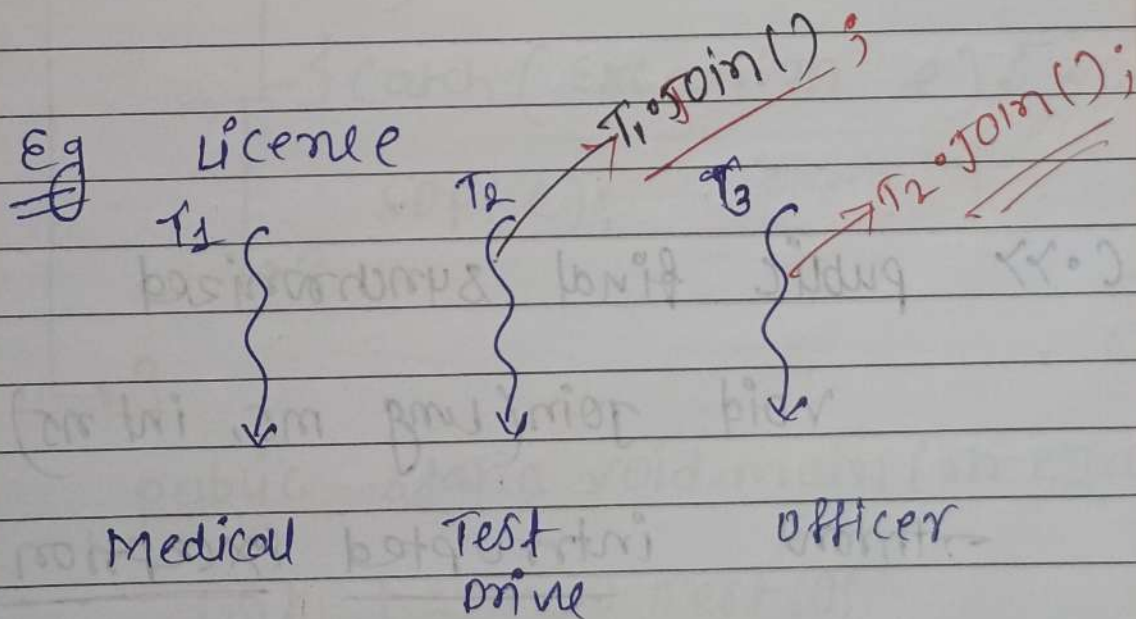
}

* join methods in java * * multithreading *

Throw interrupted exception

join() :- if a thread wants to wait for another thread to complete its task.

then we should use join() methods



T1 task complete then (T2) join
T2 task complete then $\Rightarrow$ T3 join it

* Methods :- *

9. >> public final void join() throws

InterruptedException

10. >> public final synchronized

void join(long millisec)

throws InterruptedException

11. >> public final synchronized

void join(long ms, int ns)

→ throws InterruptedException

class Test extends Thread {

public void run() {

try {

for (int i = 0; i < 5; i++) {

Sop("child thread + i");

Thread.sleep(1000);

}

} catch (Exception e) {

Sop(e);

}

}

public static void main (String args) {

Test t = new Test();

t.start();

t.join() // यहाँ पर भी मिल सके हैं


```

try {
    for(int i=0; i<5; i++) {
        sop("main thread " + i);
        Thread.sleep(1000);
    }
    vt.join();
}
catch (Exception e) {
    sop(e);
}

```

3 } methods close

3 } class close

These all three methods temporarily stop the thread execution

Difference between sleep(), yield() and join() methods

<u>properties</u>	<u>sleep()</u>	<u>yield()</u>	<u>join()</u>
<u>purpose :-</u>	if any thread does not want to perform any operation for particular time.	it stop the current executing thread provide another thread s.h. priority	if a thread want to wait for another thread to complete its <u>task</u>

Example } Timer, ppt Big-Bazar, shop, license
 Blinking, Bulb,

<u>How the thread invoke</u>	* <u>automatically</u> invoke after particular time periods	<u>Automatically</u> invoke by <u>thread scheduler</u>	* <u>Automatically</u> invoke after completion of another thread task
	* if thread is interrupted		* after completion of <u>priority time per</u>
			* if interrupted

Method

sleep(long ms)

yield()

join()

sleep(long ms, int ns)

join(long ms)

join(long ms, int ns)

Is method
overloaded

yes

no

yes

Exception

interrupted
exception

no

interrupted
exception

Is method
final

no

no

yes

Static
method

yes

yes

no

Interupt() and

Interrupted() & isInterrupted()

Methods in java

Interrupt() methods

* Thread को interrupt करवाने के लिए !

* it is used to interrupt an executing thread.

* interrupt methods will only work when the methods

is in sleeping or waiting state

* if a thread is not in sleeping or waiting state then

calling an interrupt() methods

will perform normal behavior

* $\Rightarrow$ Interrupt() methods throws *

An Exception

Interrupted Exception

* Syntax :

public void interrupt();

class test extend Thread

{

public void run()

{

try {

for(int i=1; i<5; i++) {

 sap(i)

 Thread.sleep(1000);

} catch (exception e) {

 e.printStackTrace();

}

}

public static void main (String args) {

 test t = new test();

 t.start();

 t.interrupt();

}

main thread
m. close
close close

interrupted() methods

यह check करता है कि हमारे

*→ Thread interrupt है या नहीं है।

it is used to check wheather
a thread is interrupted
or not

*→ interrupted() methods clear the

interrupted states from

(true) to (false) if thread is

interrupted

*→ इसको दो बार used करते हैं तो

(output change) कर देगा!

*→ interrupted methods will change
the (result)

Syntax

public static boolean

interrupted()

}

class Test extends Thread {

public void run() {

sop(Thread.interrupted());

try {

for (int i=1; i<=5; i++) {

sop(i);

Thread.sleep(1000);

} catch (Exception e) {

sop(e);

}

}

public static void main (String[] args)

{

Test t = new Test();

t.start();

t.interrupt();

}

check if
it

Thread of

main
Thread

isInterrupted() Methods

* isInterrupted() method is used to check whether a thread

is interrupted or not

अगर thread interrupted है तो
true return करेगा

else false return करेगा;

* isInterrupted method does not
clear the interrupted
status.

* इसको एक सै जगहा बार use करने पर
result वही आएगा same ही रहेगा,

* Syntax:

```
public boolean isInterrupted() { ... }
```


class Test Extends Thread

stop (thread interrupted)

public void run()

{ try

for (int i=1; i<=5; i++)

{ stop(1)

Thread.sleep(1000);

} catch (Exception e)

{ stop(e)

}

}

public static void main (String arg)

{ Test t = new Test();

t.start();

t.interrupt();

* synchronization in java *

*> synchronization एक process है जिसके
through हम multiple thread के
Accessibility को control करते हैं।

it is process by which we
control the accessibility of

multiple threads to a particular
shared resource.

problem which can occur without

synchronization

Data inconsistency

thread interference

** Advantage of synchronization

No data inconsistency problem

No thread interference

DisAdvantage of synchronization

* Increase the waiting time
period of thread

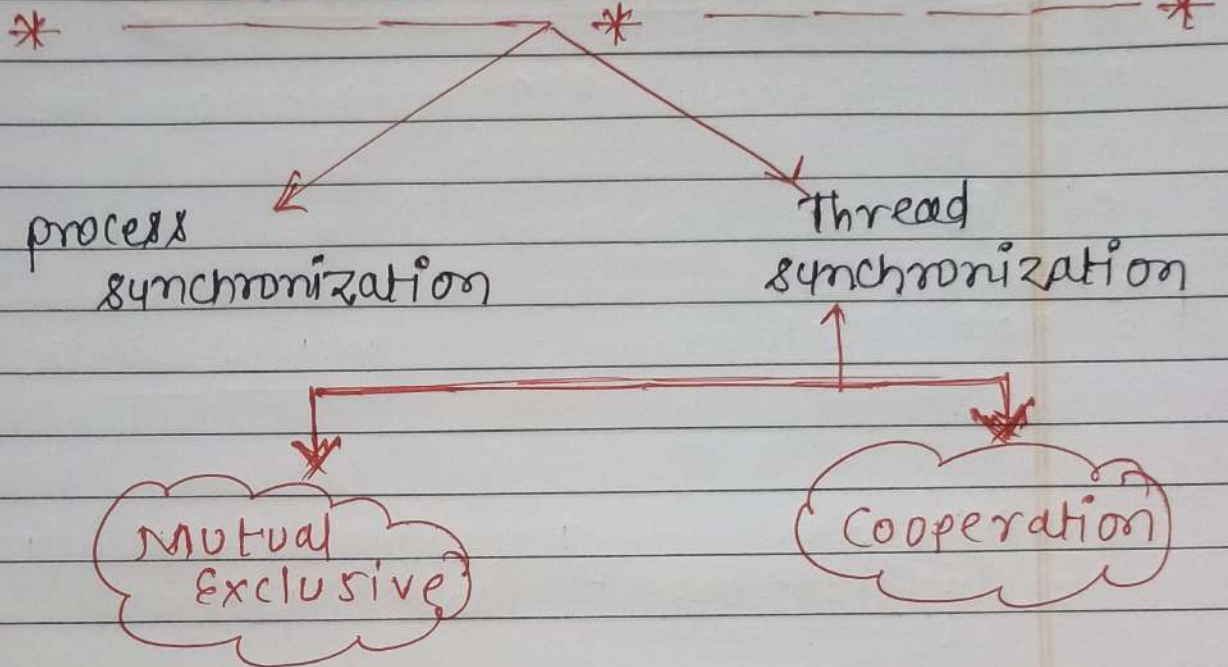
* Create performance problems

*** → But it can also be handled in

new Java (version)

By java.util.concurrent

classification of synchronization



can be Achieved
by (3) way

achieved by md
of object class

* synchronized methods

① wait()

* synchronized Block

② notify()

* Static synchronization

③ notifyAll()

Tric $\Rightarrow$ MBS

M $\rightarrow$ Method

B $\rightarrow$ Block

S $\rightarrow$ Static